Keywords: multi-agent systems • LLM agents • goal drift • reinforcement learning

Learning to Steer LLM Agents and Multi-Agent Systems

A Dual-Path Steering Agent Trained with GRPO Corrects Goal Drift and Transfers to Unseen Executors

By Leyang Shen, Yaxin Gong, et al.

arXiv:2610.22316 • October 2026

LLM-based agents—whether acting alone or in concert—are prone to goal drift: as execution unfolds step by step, the system progressively deviates from its original objective. This paper introduces a dedicated *steering agent* that monitors ongoing execution and corrects drift via two intervention paths, jointly trained with Group Relative Policy Optimization (GRPO). On information-seeking benchmarks, the steered system completes tasks more accurately and efficiently than executor-only baselines, and its policy transfers to unseen task types and executor architectures without retraining.

AT A GLANCE

Problem: Agents and multi-agent pipelines drift from the overall goal as execution steps accumulate, wasting compute and producing unintended outcomes.

Why it matters: No existing component has global oversight of execution trajectory; prompt-based corrections lack a trained policy for when and how to intervene.

Method: Dedicated steering agent with fast (cheap) and slow (expensive) intervention paths, jointly trained with GRPO and a pairwise ranking objective.

Result: Improved task completion rate and fewer total execution steps; policy transfers to out-of-domain tasks and unseen executors.

Evidence: Experiments on information-seeking single-agent and multi-agent tasks; ablations confirm both paths are necessary.

The Big Picture

Modern LLM agent pipelines are long: they involve dozens of tool calls, web searches, sub-task handoffs, and intermediate reasoning steps. Each step introduces the possibility that the agent drifts from the high-level objective—optimizing a sub-task at the expense of the whole, misinterpreting a retrieved document, or losing track of a constraint stated many steps earlier.

In multi-agent settings the problem compounds: agents optimize locally, communication between them is asynchronous, and no single agent has a global view of the pipeline's progress.

This paper's central argument is that the solution should not be to redesign every executor agent but to add a separate *steering* agent whose sole job is global oversight. The steering agent watches, decides when to act, and intervenes minimally to keep the execution on course.

Why Existing Approaches Fall Short

Prompt injection: Injecting correction instructions into the next step is brittle because it lacks a policy for deciding when or how to intervene. Over-intervention stalls progress; under-intervention allows drift to persist.

Reward shaping during training: Re-training all executors with a global objective requires access to each agent's parameters and is incompatible with off-the-shelf or proprietary executors.

Post-hoc replanning: Replanning after detecting failure is costly: it requires abandoning completed work and restarting. It cannot prevent drift mid-execution.

Core Mechanism

The steering agent operates alongside the executor(s) without modifying them. At each execution step it observes the full context and chooses among three options:

1. **Silent:** No intervention; execution continues.
2. **Fast path (low-cost):** A lightweight corrective signal, such as re-anchoring the executor to the current sub-goal or pruning off-track tool calls.
3. **Slow path (high-cost):** A full re-planning step or a coordinator message to all agents, for novel or complex deviations.

Both paths and the routing decision are trained jointly, so the steering agent learns not only how to correct but also when each type of correction is warranted.

Technical Formulation

Let s_t be the execution state at step t and a_t^{exec} the action the executor would take absent intervention. The steering agent produces $\delta_t \in \{\text{silent}, \text{fast}, \text{slow}\}$ and a steering signal σ_t. The executor's effective action is:

$$a_t = f(a_t^{\text{exec}},\ \sigma_t)$$

where f merges the executor's action with the steering signal. The GRPO objective for the steering agent is:

$$\mathcal{J}_{\text{steer}}(\theta) = \mathbb{E}\left[\sum_t (R_{\text{final}} - \lambda_{\text{cost}} \cdot \mathbf{1}[\delta_t = \text{slow}]) \cdot \log \pi_\theta(\delta_t, \sigma_t \mid s_t)\right]$$

with λ_{cost} penalising slow-path invocations, pushing the policy to use the cheap fast path whenever sufficient.

A supplementary pairwise ranking loss compares trajectory pairs (same task, different steering decisions) and penalises whichever produced the worse outcome:

$$\mathcal{L}_{\text{rank}}(\theta) = \mathbb{E}_{(\tau^+,\tau^-)}\left[\max\left(0,\ 1 - \log \frac{\pi_\theta(\tau^+)}{\pi_\theta(\tau^-)}\right)\right]$$

The final training objective is $\mathcal{J}_{\text{steer}} - \alpha\mathcal{L}_{\text{rank}}$.

Learning or Inference Procedure

1. Collect paired trajectories: one with the steering agent active, one without (or with a different steering decision).
2. Annotate outcomes; form pairwise preference labels for the ranking loss.
3. Train the steering agent with the combined GRPO + ranking objective.
4. At inference, the steering agent runs alongside the executor(s); no modification to executors is required.
5. Intervention frequency is controlled post-training via λ_{cost}, enabling efficiency-accuracy trade-offs.

What the Guarantee Says

The pairwise ranking objective provides a contrastive guarantee: the steering policy converges to prefer interventions whose outcomes are better in expectation than the counterfactual of no intervention. The GRPO component constrains the policy to stay close to the reference model, preventing exploitation of the steering interface. Together, the objectives bound the expected cost overhead of incorrect interventions relative to executor-only execution.

Experimental Findings

- **Single-agent information-seeking:** The steering agent improves task completion rate and reduces total execution steps compared to the executor-only baseline, prompt-injection correction, and replanning-on-failure.
- **Multi-agent pipelines:** Gains are larger in multi-agent settings, where drift compounds across agents; the steering agent provides global coordination that individual agents lack.
- **Transfer to out-of-domain tasks:** The steering policy, trained on information-seeking tasks, transfers to document QA and planning tasks without any fine-tuning.
- **Transfer to unseen executors:** Executors not encountered during training are steered effectively, confirming that the policy learned general drift-correction rather than executor-specific heuristics.

Ablations and Interpretation

Fast-path only: Loses on novel drift patterns requiring deep re-planning; confirms the slow path is necessary for non-routine corrections.

Slow-path only: Higher cost per step with marginal accuracy gain over fast-only; the fast path handles the majority of cases cheaply.

Without pairwise ranking (GRPO alone): Noisier intervention decisions; the steering agent more often invokes the slow path when the fast path would have sufficed, indicating the ranking loss sharpens the routing decision.

Controllable intervention: Sweeping λ_{cost} traces a smooth Pareto curve between task accuracy and intervention cost, confirming the design supports post-training trade-off adjustment.

Reference

Leyang Shen, Yaxin Gong, et al. **Learning to Steer LLM Agents and Multi-Agent Systems**. arXiv:2610.22316, October 2026. https://arxiv.org/abs/2610.22316